Sistemas Operativos Avanzados (1123) - Primer parcial

1er cuatrimestre - Día Martes – 28/09/2021

Nombre y Apellido:

Nro. Grupo:

Día de cursada:

DNI:

Preguntas:

1. Teniendo en cuenta que la previsibilidad es una característica importante en STR: ¿Qué tipo de planificador la utiliza mejor?
2. ¿Cuáles son los 3 requisitos que debe proveer un SOTR?
3. Explique porque no es conveniente usar las placas Arduino como un producto final.
4. ¿Cómo se compilan los programas que se ejecutan en Arduino UNO? Explique el concepto.
5. Si un SE, implementado en Raspberry PI 4, solo utiliza Bluetooth para comunicarse con un smartphone. ¿Considera que es internet de las cosas?
6. ¿Qué ventajas tiene utilizar temporizadores versus la función delay()?
7. ¿Qué patrón de diseño de FSM implementó en su código del TP1? ¿Por qué?
8. ¿Qué es PWM? Explique de qué forma lo utilizó en su Trabajo Práctico.
9. ¿Qué cambios son requeridos en su Trabajo Práctico, para agregar un pulsador, que funciona con interrupción externa, para activar o desactivar el logueo por el monitor serial?
10. ¿Por qué se le pide en el TP1 que se agregue una fuente regulable extra para alimentar los sensores y actuadores?

1. Teniendo en cuenta que la previsibilidad es una característica importante en STR: ¿Qué tipo de planificador la utiliza mejor?

El tipo de planificador que mejor utiliza la previsibilidad son los planificadores con control de admisión, de esta manera sabiendo de antemano cuanto lleva ejecutar la tarea y cuando es el deadline, puedo saber si voy a terminar la tarea a tiempo o no, garantizando su cumplimiento El beneficio de utilizar este tipo de planificador es que se puede detectar situaciones de sobrecarga con anticipación y saber que tareas podemos lograr, llegando asi al concepto de previsibilidad.

Nota: Este tipo de planificador nunca se presenta en la PPT, pero hace referencia a planificadores que estadísticamente pueden predecir comportamientos.

Buscar la diferencia entre planificadores basados en prioridades y planificadores con control de admisión (enfocados en deadlines). RMS vs EDF

Respuesta ChatGPT:

Prompt: Dado los planificadores: Planificación según periodo, Planificación por prioridad monótona en tasa y Planificación por prioridad en finalización de plazo ¿Que planificador es mejor?

Los planificadores mencionados son ampliamente utilizados en sistemas de tiempo real y cada uno tiene ventajas en diferentes contextos. Vamos a analizar brevemente cada uno para determinar cuál puede considerarse mejor según el tipo de sistema:

1. Planificación según periodo (Rate Monotonic Scheduling - RMS):

- Descripción: Asigna prioridades estáticas basadas en el período de cada tarea. Las tareas con periodos más cortos (mayor frecuencia de ejecución) tienen mayor prioridad.
- Ventajas: Es óptimo para sistemas de tiempo real periódicos bajo ciertas condiciones (sistemas de tareas independientes, preempción y tiempos de ejecución conocidos). Si el sistema está por debajo del 69.3% de utilización del procesador, RMS garantiza que las tareas cumplirán sus deadlines.
- Desventajas: No es eficiente si el sistema tiene una alta utilización de la CPU. Además, no es óptimo en comparación con otros algoritmos como EDF (no garantiza el cumplimiento de deadlines cuando la CPU está altamente utilizada).

2. Planificación por prioridad monótona en tasa (Rate Monotonic Scheduling - RMS):

- Descripción: Es el mismo concepto que la planificación según período. Las prioridades se asignan de forma monótona en función de la tasa o frecuencia de ejecución de las tareas, es decir, las tareas más rápidas tienen mayor prioridad.
- Ventajas y desventajas: Igual que el punto anterior, ya que ambos se refieren a RMS.

3. Planificación por prioridad en finalización de plazo (Earliest Deadline First - EDF):

- Descripción: EDF asigna prioridades dinámicas según el plazo más cercano. La tarea con el deadline más próximo recibe mayor prioridad.
- Ventajas: Es óptimo en la utilización de la CPU para sistemas de tiempo real. Mientras la utilización del procesador no supere el 100%, EDF garantiza que todas las tareas

cumplirán sus deadlines. Tiene una alta eficiencia y es más flexible que RMS en sistemas sobrecargados.

- Desventajas: EDF es más complejo de implementar y no es tan adecuado para sistemas con tareas de alta prioridad que necesitan ser ejecutadas de inmediato, ya que las prioridades cambian dinámicamente.

Comparación de los Planificadores

- RMS (Planificación según periodo o por prioridad monótona en tasa) es más sencillo y efectivo en sistemas con cargas moderadas, donde las tareas tienen periodos fijos y la CPU no está sobrecargada. Es ideal para sistemas donde la previsibilidad es crítica y el costo de implementación debe ser bajo.
- EDF (Planificación por prioridad en finalización de plazo) es más eficiente en términos de utilización de la CPU y ofrece mejores garantías para cumplir con los deadlines en un sistema con tareas más variables y carga alta. Sin embargo, es más complicado de implementar y requiere un análisis más detallado de los deadlines dinámicos.

¿Cuál es mejor?

- EDF es, en general, más flexible y eficiente, ya que garantiza que las tareas se completen dentro del deadline siempre que la CPU esté suficientemente dimensionada. Por lo tanto, es mejor en sistemas con alta utilización de CPU o con cargas variables.
- RMS es más sencillo y predecible, siendo mejor para sistemas donde las tareas son periódicas y el diseño debe ser simple, a costa de una menor eficiencia en la utilización de la CPU.

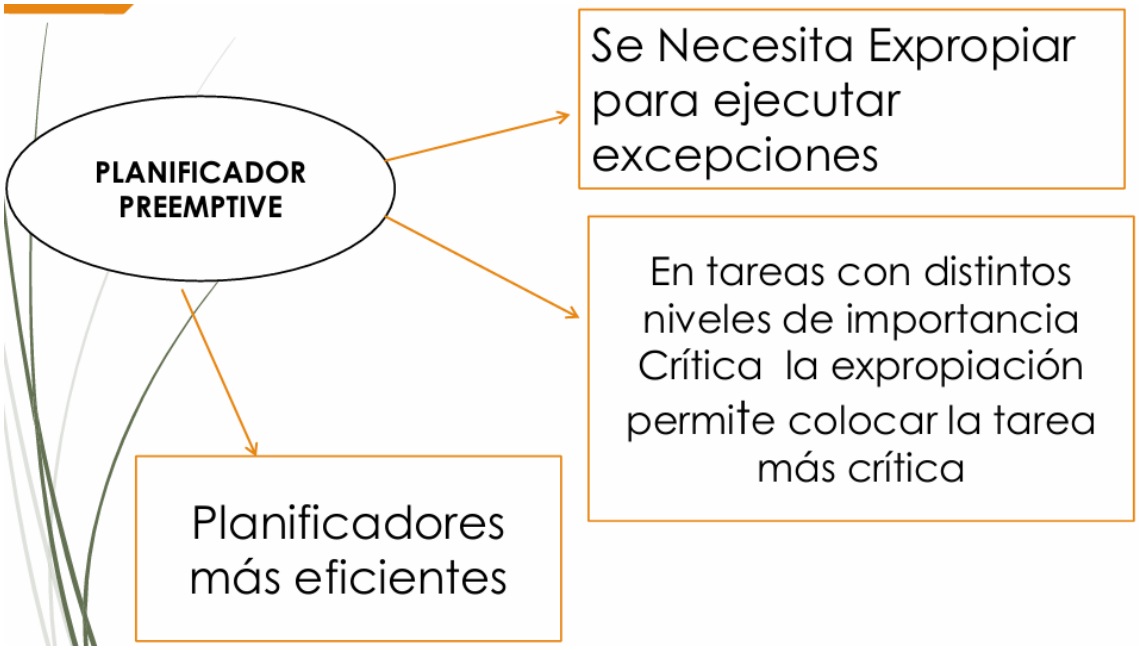
La elección depende del tipo de aplicación. Para un sistema de tiempo real estricto, con periodos bien definidos y cargas predecibles, RMS puede ser suficiente. Para un sistema de tiempo real dinámico, con mayor variabilidad y utilización intensiva del procesador, EDF sería preferible.

2. ¿Cuáles son los 3 requisitos que debe proveer un SOTR (Sistema Operativo de Tiempo Real)?

- Planificador preemptive basada en prioridades: Las tareas más importantes tiene mayor prioridad respecto a las menos importantes. Si un proceso con mayor prioridad pasa a la cola de listos, lo pongo a ejecutar.
- Kernel apropiativo: Permite desalojar al kernel y de esta manera garantizar que un proceso de mayor prioridad no sea bloqueado por uno de menor prioridad que utilice la rutina del Kernel.
- Latencia mínima: Dado que un RTS se encuentra normalmente esperando eventos externos, hay que responder tan rapido como se pueda, esto es disminuir la latencia de eventos (el tiempo entre que un evento ocurre y el RTS lo atiende). Hay dos latencias que afecta a estos sistemas:
    - Latencia de Interrupción: Es el tiempo entre que llega la interrupción hasta la rutina de atención de interrupción (RAI)
    - Latencia de Dispatch: Tiempo que tarda en RTS en resolver los conflictos del dispatcher y despachar nuevos procesos (parar un proceso y arrancar otro).

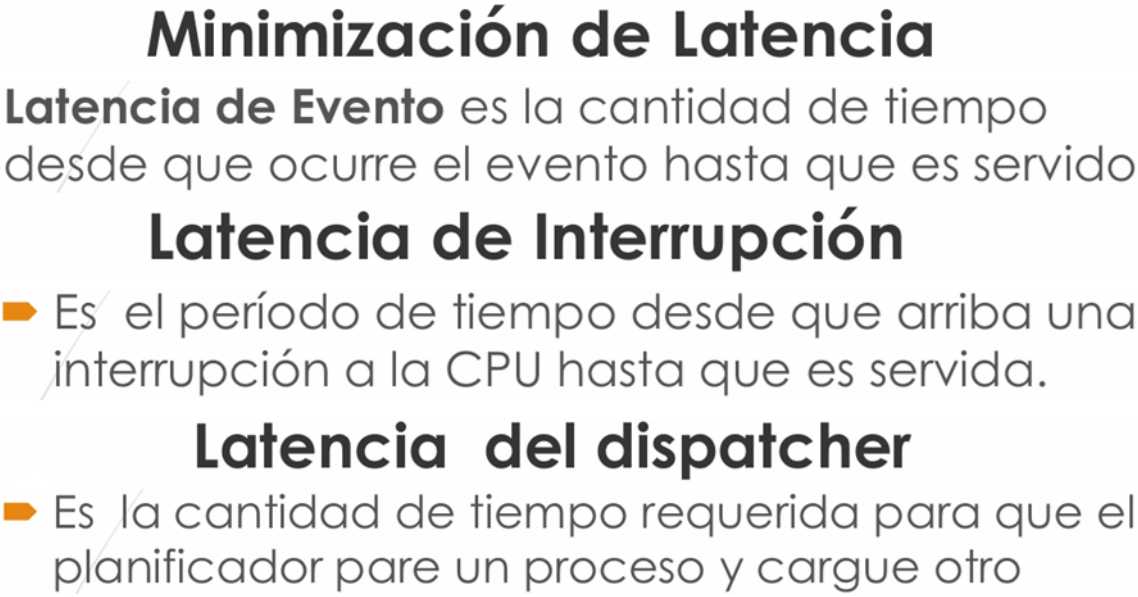
En general, un Sistema Operativo en Tiempo Real debe proveer :

- Planificación por **Prioridades**, **Preemptive**
- **Kernel Preemptive**
- La **latencia** debe ser **minimizada**



Kernels Apropiativos

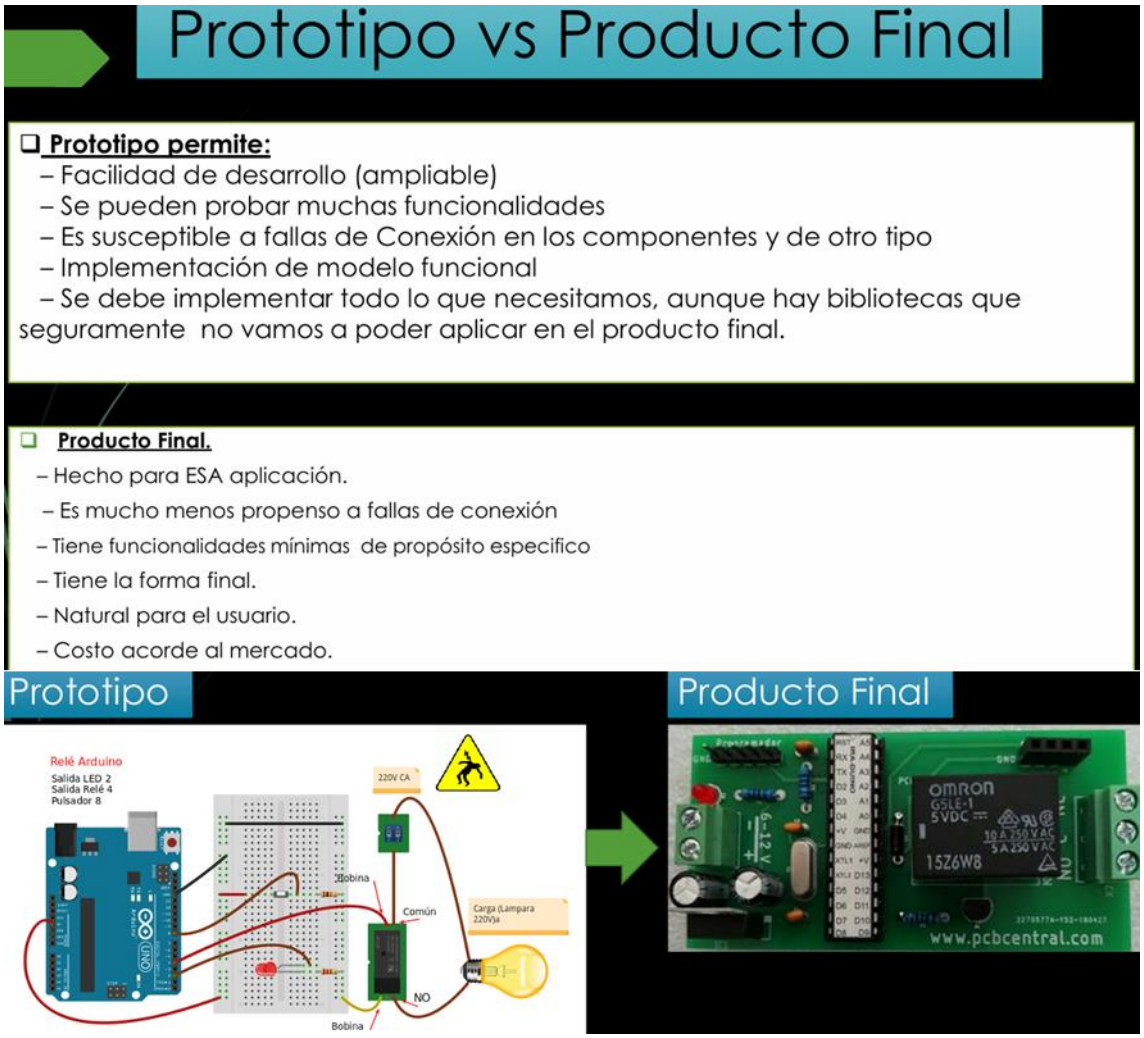
- Permiten desalojar un proceso ejecutando en modo Kernel
- Reduce los tiempos de espera por recursos
- Puntos de desalojo o mecanismos de sincronización



3. Explique porque no es conveniente usar las placas Arduino como un producto final.

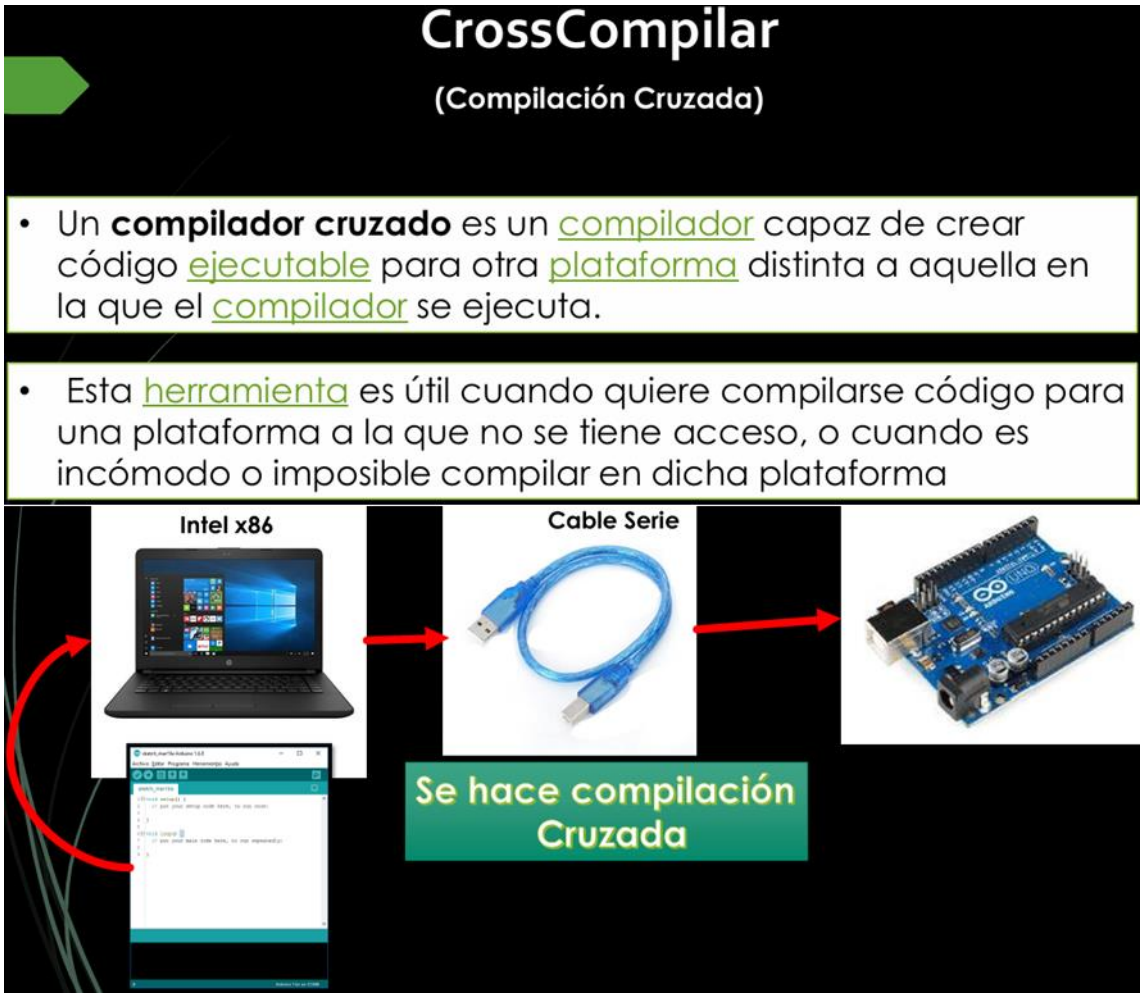
Las placas de Arduino son placas de desarrollo que se utilizan generalmente para realizar prototipos. El prototipo es muy endeble: tiene cables que se pueden mover o desconectar y no necesariamente se utilizan todos los pines para el prototipo.

Es por esto que para un producto final es lo recomendable armarlo en un PCB: ponerle un microcontrolador y soldar las conexiones. Además, si es un producto final a propósitos comerciales, está prohibido utilizar placas Arduino para la venta por ley comercial.



4. ¿Cómo se compilan los programas que se ejecutan en Arduino UNO? Explique el concepto.

Se compilan en una computadora por ejemplo de escritorio y se escriben en la memoria ROM del Arduino mediante el puerto serial. El concepto que se utiliza es el de compilación cruzada, esto quiere decir que yo compilo en mi computadora de escritorio para otra arquitectura diferente a la de mi PC. A su vez estoy compilando un programa que no se ejecutará en mi PC si no que lo hará en el Arduino.



5. Si un SE, implementado en Raspberry PI 4, solo utiliza Bluetooth para comunicarse con un smartphone. ¿Considera que es internet de las cosas?

No, para que sea considerado un IoT es necesario el procesamiento en la nube. Esto quiere decir que la información recibida del embebido viaje a través de un gateway a un servicio de terceros y procese dicha información. En todo caso, sería un Machine to Machine, ya que solo se realiza la transmisión de datos a través de Bluetooth, por lo tanto, nunca utiliza internet, lo cual hace que los datos nunca se transmitan a la nube.

6. ¿Qué ventajas tiene utilizar temporizadores versus la función delay()?

Si se utiliza delay, estaria bloqueando el sistema embebido y eso ocasionaría problemas como, por ejemplo, si llega información de un sensor, se perdería posibles eventos. Por lo tanto, una ventaja de utilizar temporizadores es que no se utilizan funciones bloqueantes.

7. ¿Qué patrón de diseño de FSM implementó en su código del TP1? ¿Por qué?

El patron de diseño que se implementó en el proyecto es patron orientado a instrucciones condicionales. Esto quiere decir que dado un determinado estado del sistema embebido voy a estar entrando a los distintos cases donde, de forma anidada, se encuentran los eventos dentro de otro case.

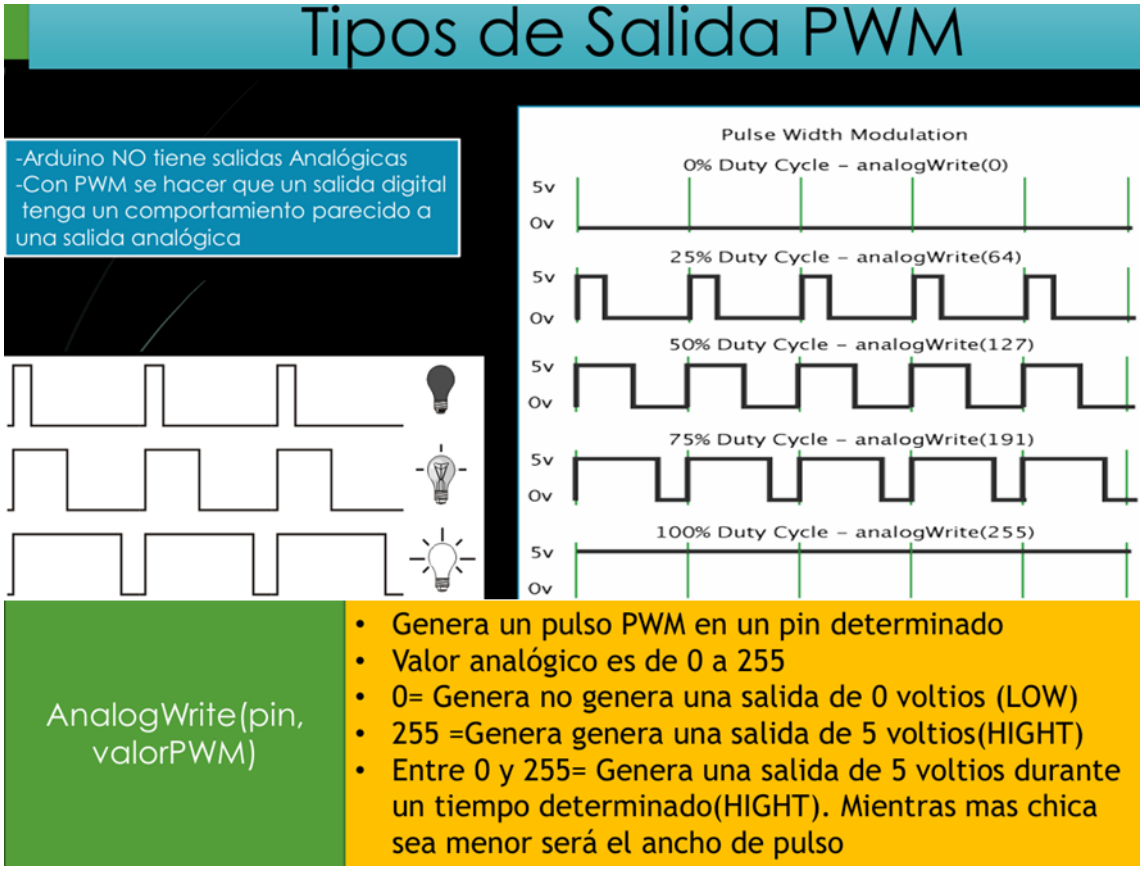
Se implemento este patron de diseño, no por nada en particular, sino porque lo consideramos más entendible y organizado que el patron de diseño de FSM basado en tablas / matrices. ¿Estaria bien esta respuesta?

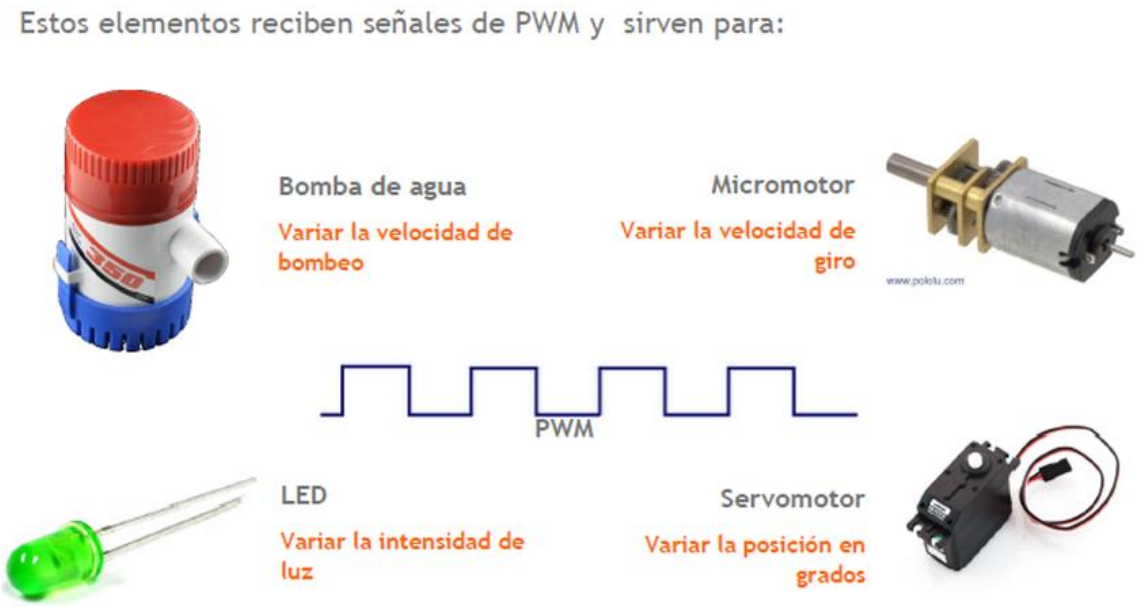
8. ¿Qué es PWM? Explique de qué forma lo utilizó en su Trabajo Práctico.

PWM o Modulación por Ancho de Pulso es una técnica que consiste en aplicarle a una señal digital un comportamiento que tendría una señal analógica. Es utilizada para controlar la cantidad de energía entregada a una carga o dispositivo mediante la variación del ancho de los pulsos en una señal periódica. PWM funciona de la siguiente manera: para enviar una señal analógica que se encuentre entre 0 y 255, se utiliza el concepto de ancho de pulso. El ancho de pulso corresponde a un porcentaje del duty cycle en el que se envía una señal HIGH, mientras que en el resto del duty cycle se envía una señal LOW.

En el caso del Trabajo Practico, se utilizó para regular la velocidad de un motor de corriente continua mediante la utilización de un Puente H, este último tiene un pin que se conecta al Arduino Uno para aplicar esta técnica.

Si bien se aplicó un criterio para que esta sea una velocidad constante, quizás al pasarlo al físico será necesario regular esta velocidad. De momento, solo prende y apaga el motor mediante un analogWrite().





9. ¿Qué cambios son requeridos en su Trabajo Práctico, para agregar un pulsador, que funciona con interrupción externa, para activar o desactivar el logueo por el monitor serial?

En cuanto al hardware, se deberá conectar el pulsador a unos de los pines del Arduino que soporte interrupciones, estos pines son el pin 2 y 3. Por lo tanto se tendrá que conectar uno de estos pines.

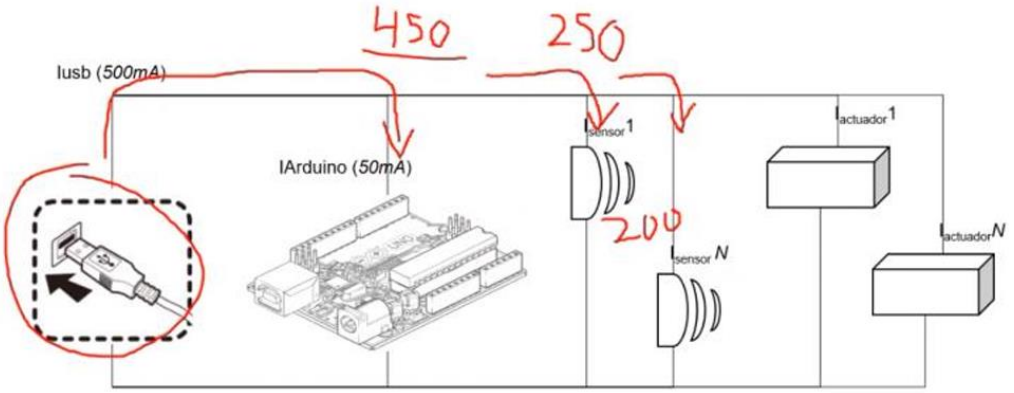
En cuanto al software, se deberá declarar el pin del pulsador como INPUT mediante `pinMode()`. Tambien es necesario declarar la interrupción con `attachInterrupt()`, el cual voy a estar pasando como parametros, el pin del pulsador, el manejador de la interrupción (que se deberá programar), y en qué tipo de flanco quiero que se genere la interrupción (bajo, alto, flanco ascendente o descendente).

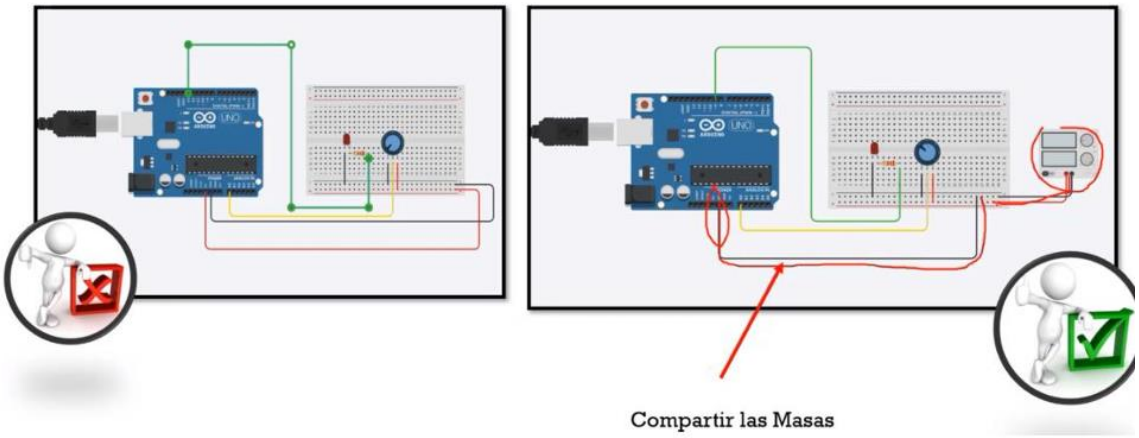
Y para finalizar, en el manejador de la interrupción que se declaró, se debe programar la lógica para activar y desactivar el logueo por el monitor serial. Esto puede (o debe) desembocar en los eventos correspondientes: uno de activación y otro de desactivación del monitor serial.

10. ¿Por qué se le pide en el TP1 que se agregue una fuente regulable extra para alimentar los sensores y actuadores?

Se recomienda una fuente regulable externa ya que, si lo conectamos directamente del Arduino, podría llegar a un punto que algunos sensores o actuadores no puedan ser alimentados porque no le va dar el amperaje que requiere cada uno de estos.

Se pide que se agregue una fuente regulable extra para alimentar los sensores y actuadores, ya que algunos consumen más que otros y Arduino no es capaz de alimentar a todos los sensores. Sin esta fuente puede pasar que el sistema comience a funcionar incorrectamente, probando valores de sensores que no deben pasar, o saltos de codigo. Con el agregado de la fuente externa, el circuito alcanza los 500 mA y los componentes funcionan correctamente.





Compartir las Masas